

Course: CSN 405 – Network Programming
LAB 4 – Multi Threading

Student name	<i>Write your name here</i>
Student ID	<i>Write your ID here</i>

Question 1

Write a Python program that implements a TCP server and a client. Each client should:

1. Send a message to the server.
2. The server should reverse the message, convert it to uppercase, and append the client's IP address and port to the response.
3. Send the processed message back to the client.

Requirements:

- The server should log the communication details (client IP, port, and message exchanged).
- The client should display the server's response and exit gracefully.

Example Communication:

- Client sends: "hello world"
- Server responds: "DLROW OLLEH (127.0.0.1:12345)"

Question 2

Design and implement a Client-Server system that uses TCP sockets to do the following:

Client identification:

- When the Client starts, it connects to the Server.
- The Client sends its name to the Server (for example: Client1).
- The Server stores the following information for the entire session:
 1. Client name
 2. Client IP address
 3. Client port number

Client side:

- After sending its name, the Client enters an infinite loop.
- In each loop, the Client reads a line of input from the user.
- The user input must be in the following format:

number operator number

Ex:	13 + 7	3.24 - 45	4.5 / -6
------------	--------	-----------	----------

- If the input format is **invalid**, the Client displays a warning message.
- If the input format is **valid**, the Client sends the input string to the Server.

Server side:

- The Server runs in an **infinite loop** waiting for messages from the Client.
- When a request is received, the Server:
 1. Prints the received message
 2. Performs the requested arithmetic operation
 3. Sends the result back to the Client as a string
- The Client displays the results received from the Server.

Example Communication:

Server side:

```
[SERVER] The server is listening ...  
Client connected: Client1 from 127.0.0.1:49942  
Received from Client1: 3.24 - 45  
Received from Client1: 4.5 / -6  
Received from Client1: 0 / 6  
Received from Client1: 6 / 0
```

Client side:

- Valid:

```
Enter arithmetic expression (e.g., "12 + 42") or "quit" to exit: 3.24 - 45  
Result: -41.76  
Enter arithmetic expression (e.g., "12 + 42") or "quit" to exit: 4.5 / -6  
Result: -0.75
```

```
Enter arithmetic expression (e.g., "12 + 42") or "quit" to exit: 13 + 7  
Result: 20.0  
Enter arithmetic expression (e.g., "12 + 42") or "quit" to exit: quit
```

- Invalid:

```
Enter arithmetic expression (e.g., "12 + 42") or "quit" to exit: 6 / 0  
Result: Error: Division by zero
```

Question 3:

Extend the system from Question 2 so that the Server can handle multiple Clients at the same time.

- The Server must use **multi-threading**.
- Each Client connection (via TCP socket) is handled by a separate thread.
- For every connected Client, the Server provides the same service described in
 1. Receive client name (id)
 2. Process arithmetic requests
 3. Send results back to the correct Client

Students must:

- Briefly explain your code
- How multi-threading works in this case

Hint:

while True:

#Each accept creates a new conn socket for the client.

conn, addr = server_sock.accept()

t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)

t.start()

handle_client: function → send and receive the data

Example Communication:

```
New thread created for client at ('127.0.0.1', 59128)
Client connected: Client1 from 127.0.0.1:59128
New thread created for client at ('127.0.0.1', 56076)
Client connected: Client2 from 127.0.0.1:56076
□
```

Question 4: TCP True/False Quiz using multi-threading for multiple clients

Server Implementation

Create a TCP server that listens to localhost, port 5050.

Prepare a list of at least 3 True/False questions:

- “Port 80 is used for HTTP.”
- “TCP is a connection-oriented protocol.”
- “UDP guarantees delivery of packets.”
- “Socket needs a tuple HOST and PORT”
- “IPv4 is the only IP can be assigned for computer”

When all questions are answered: send “Quiz finished!” to client

If the client sends quit or exit, the server should close the connection

When a client connects:

- Send the quiz questions to the client one by one

- For each question:
 - Receive the client's answer ("true" or "false")
 - Check whether the answer is correct
 - Send one of the following responses:
 - "Yes, it's correct!" if the answer is correct
 - "No, it's wrong!" if the answer is incorrect

Client Implementation

- Connect to the server on localhost, port 5050
- Receive each question from the server
- Prompt the user to enter "true" or "false"
- Send the answer to the server
- Display the server's response
- Continue until all questions are finished or the user enters "quit" or "exit"

Note: using threading for handle multi clients